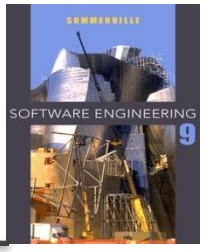
ASD27
Advanced Software Development

Adapted from slides by Ian Sommerville and Edwin Blake

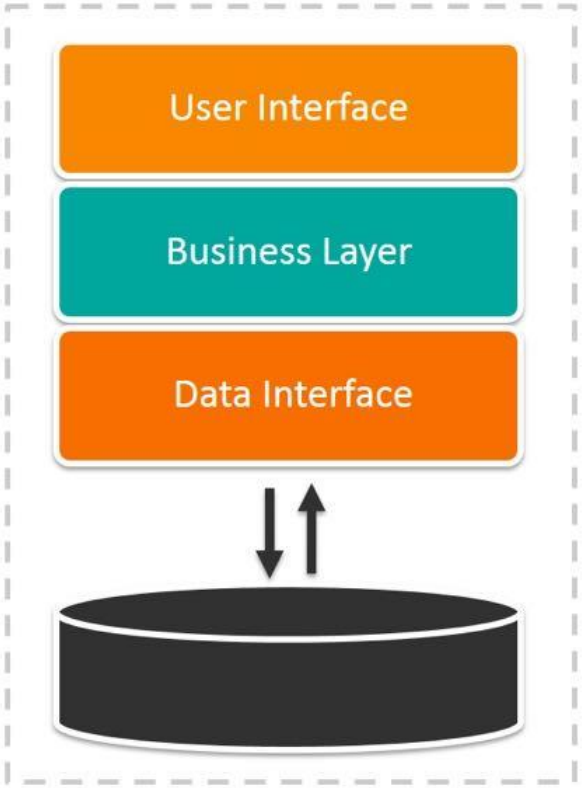
CSC3003S

Topics covered

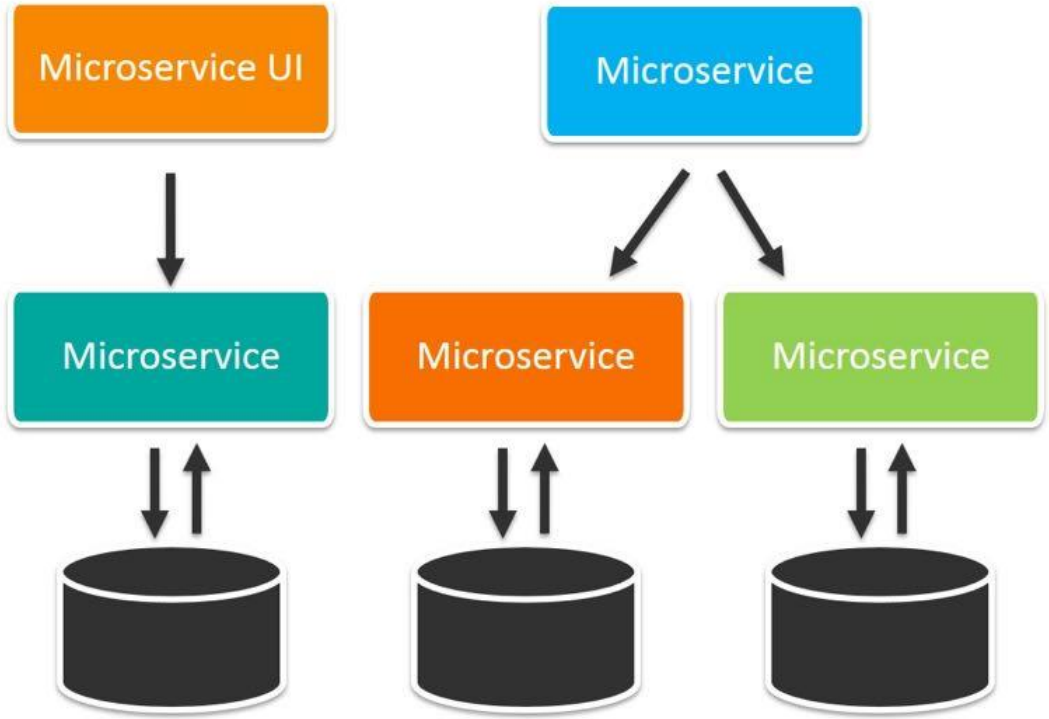


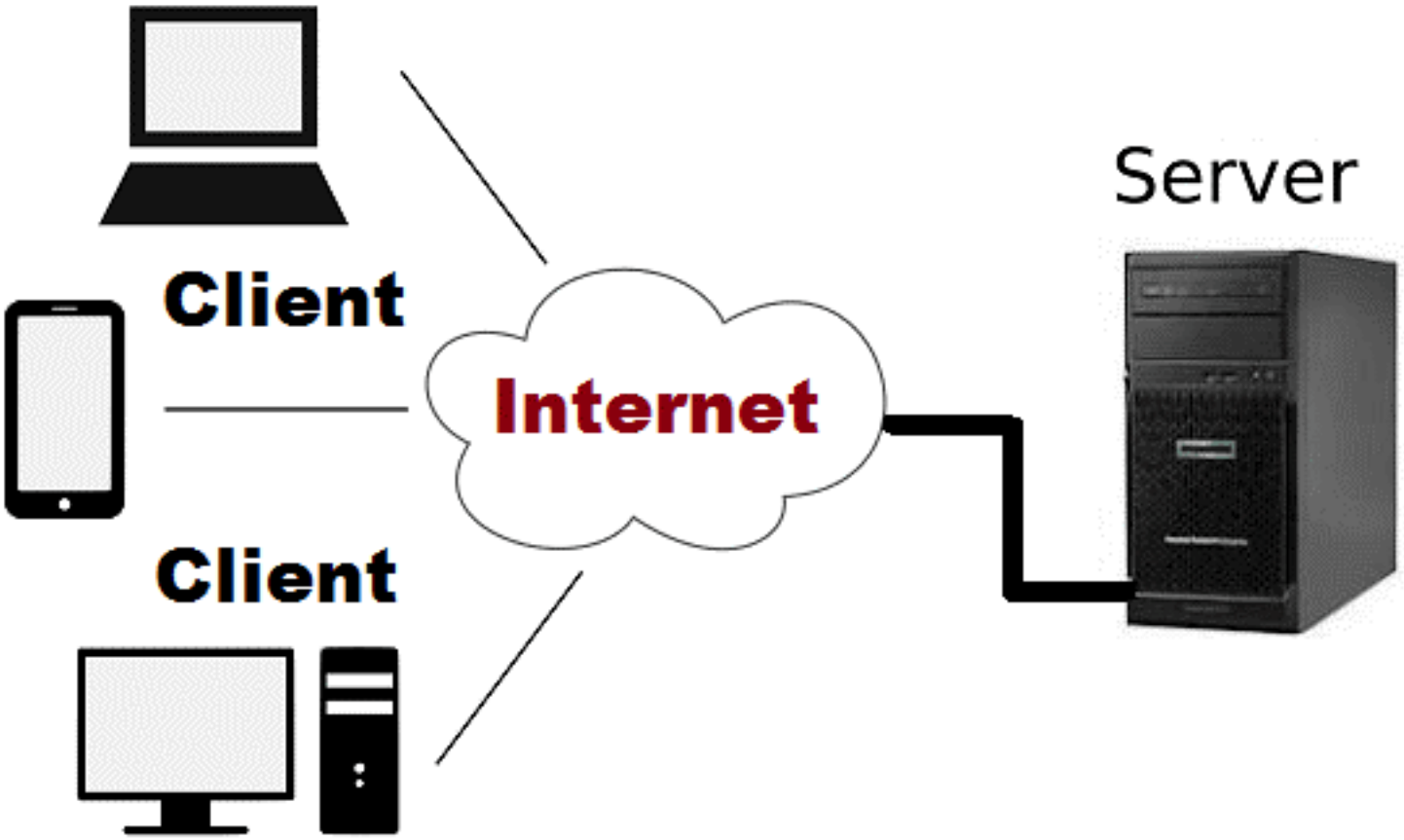
- ✧ Services
- ✧ Service engineering
- ✧ Software development with services

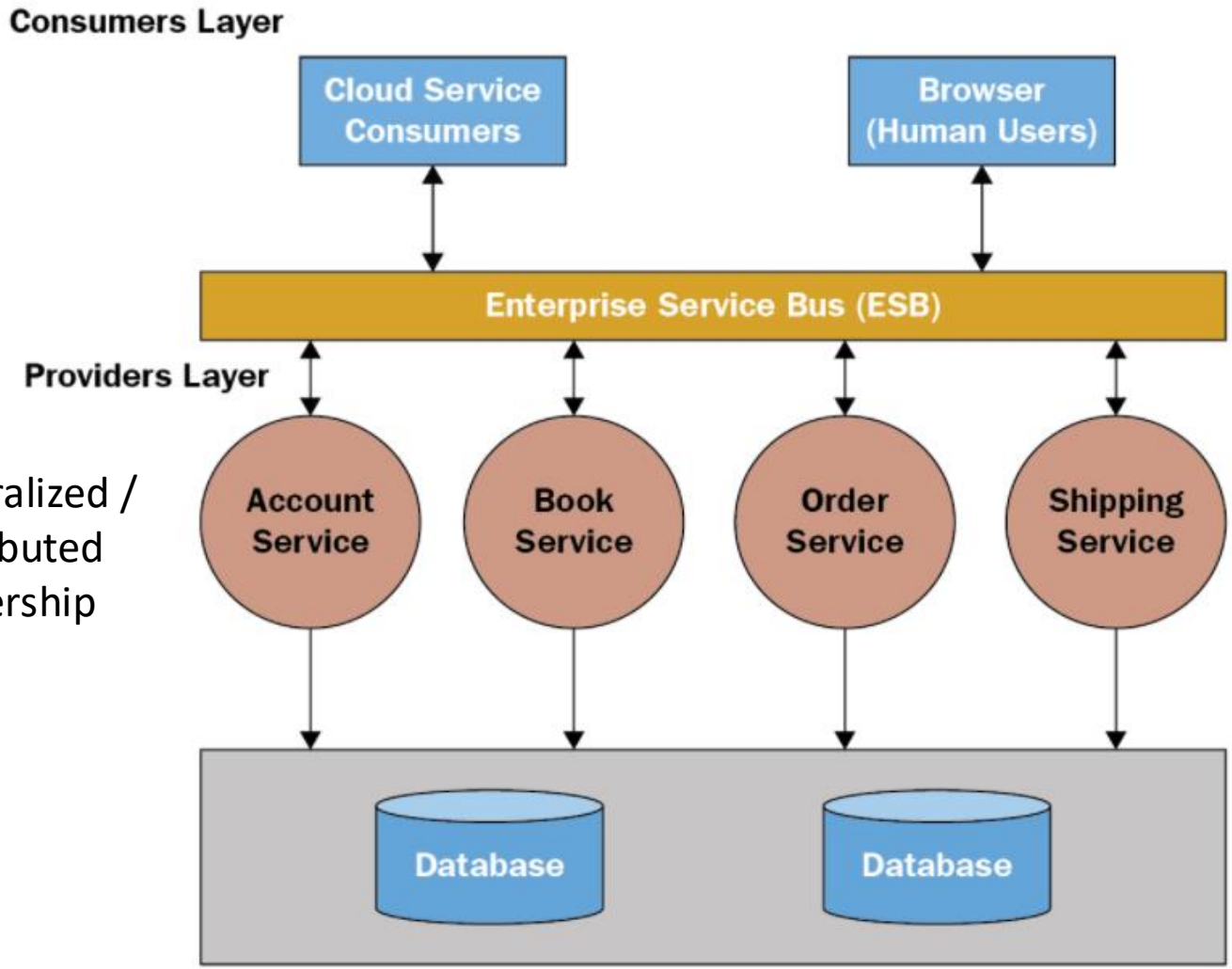
Monolithic Architecture



Microservices Architecture




Server
Client
Internet
Client

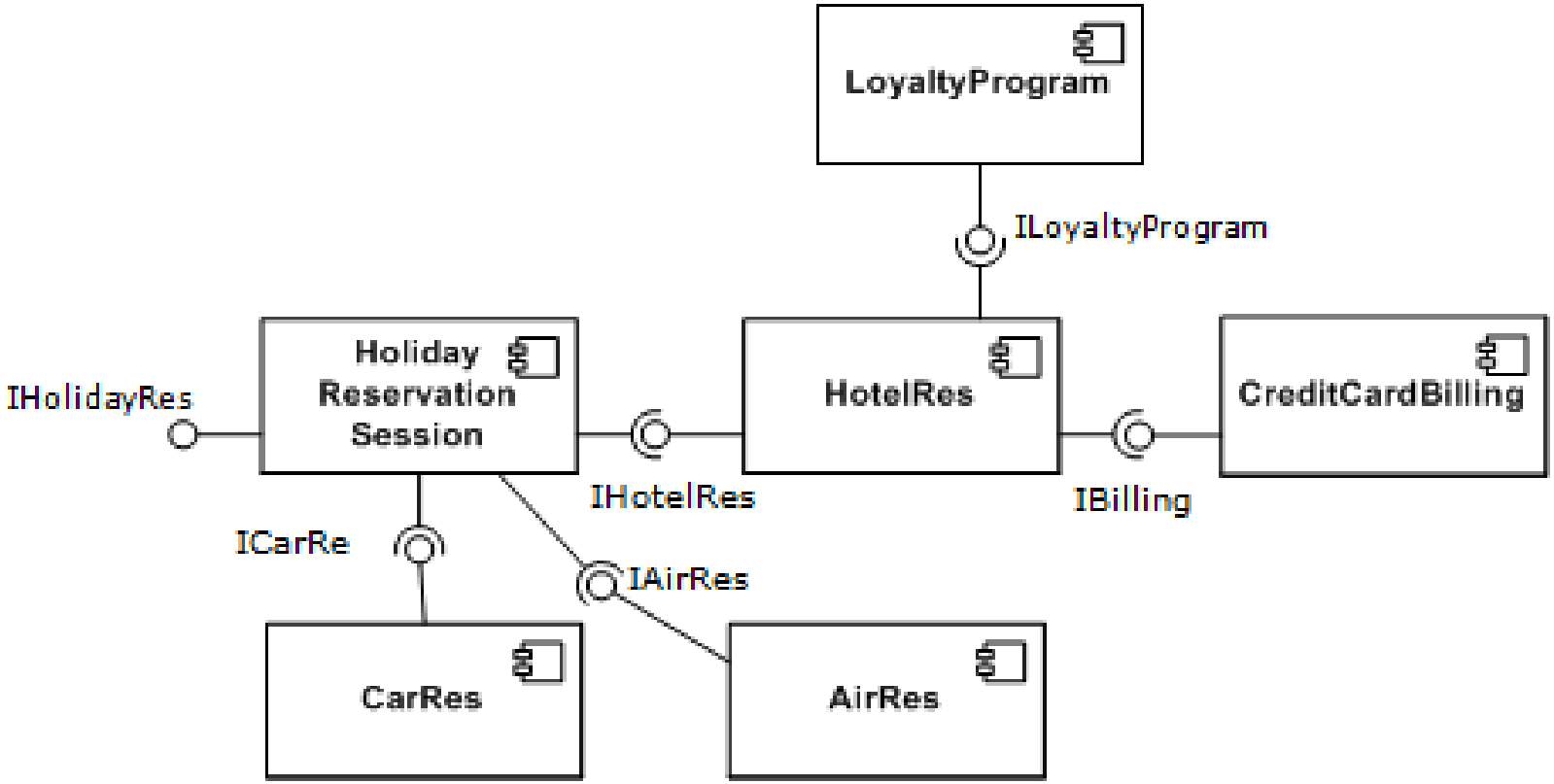


Centralized / distributed ownership

Services talk to clients using web services such as REST /SOAP

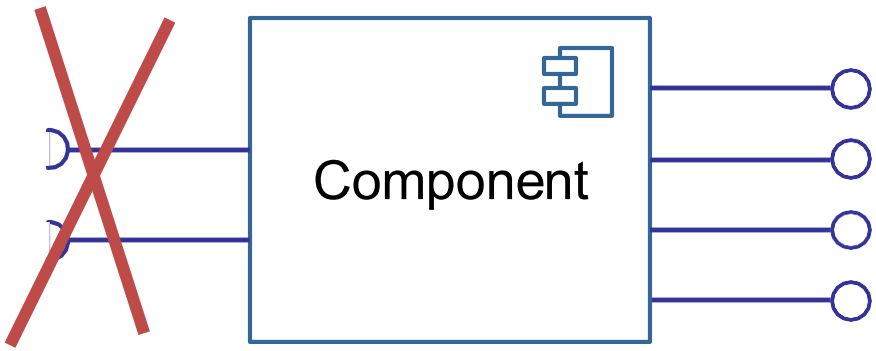
Heterogenous systems

Example: holiday-reservation system

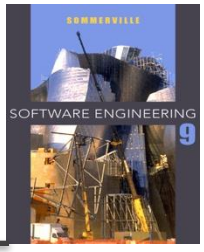


Service Oriented Architecture (SOA)

- an architecture design pattern
- independent of the applications using it
- stateless ?



Service-oriented software engineering

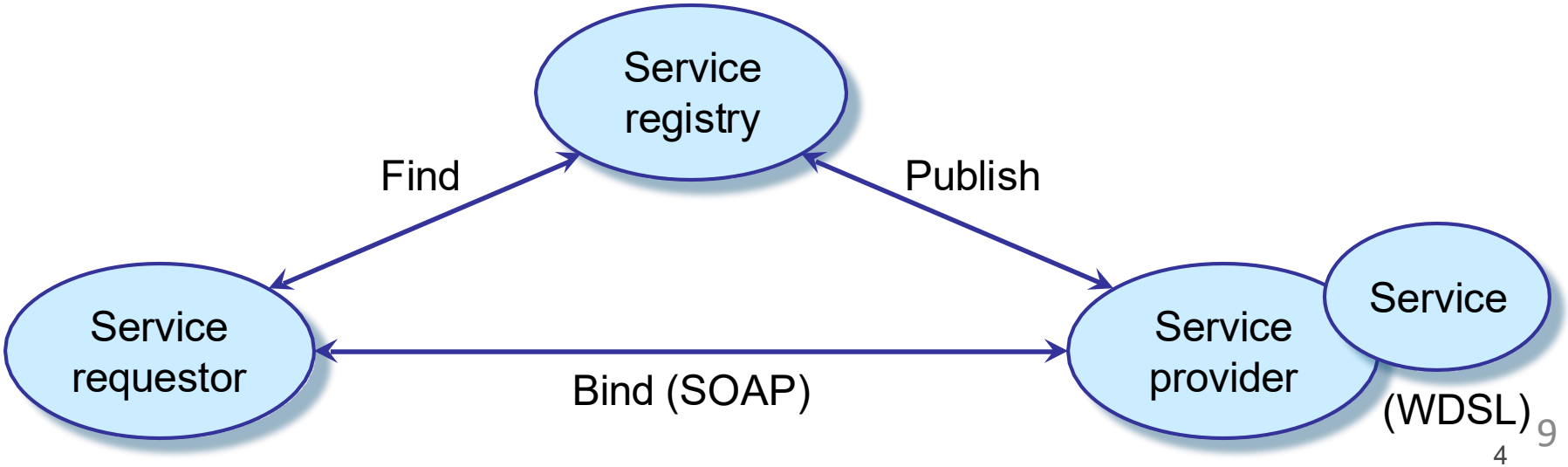


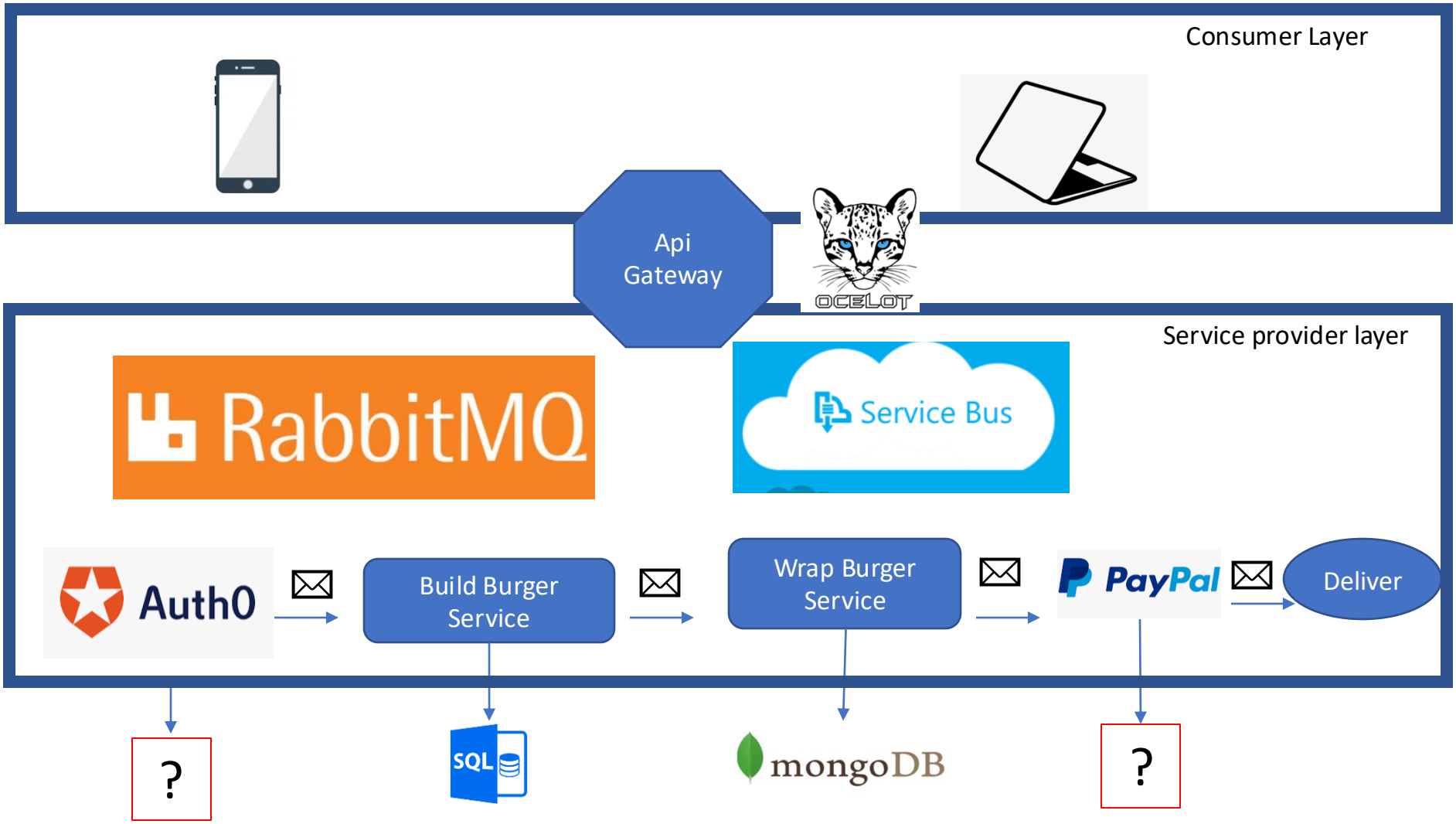
- **Service engineering**
  - development of dependable, reusable **services**
  - **for reuse**

- **Software development** with services
  - software where services are the main components
  - **with reuse**

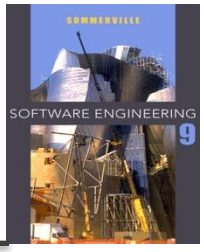
Service-oriented architectures

- A means of developing distributed systems where the components are stand-alone services
- Services may execute on different computers from different service providers
- Standard protocols have been developed to support service communication and information exchange



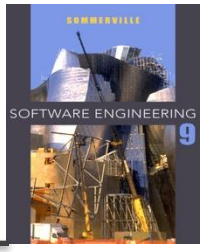

Consumer Layer
Api
Gateway
OCELOT
Service provider layer
RabbitMQ
Service Bus
Auth0
Build Burger
Service
Wrap Burger
Service
PayPal
Deliver
?
SQL
mongoDB
?

Key standards



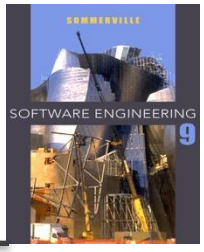
- SOAP
  - A message exchange standard that supports service communication
- WSDL (Web Service Definition Language)
  - This standard allows a service interface and its bindings to be defined
- WS-BPEL
  - A standard for workflow languages used to define service composition

SOAP



- ✧ Simple Object Access Protocol
- ✧ XML as data store
- ✧ HTTP as transport
- ✧ SOAP protocol has layers
  - message format
  - Message Exchange Patterns (MEP)
  - underlying transport protocol bindings
  - message processing models
  - protocol extensibility
- ✧ Work like remote procedure calls (RPC)

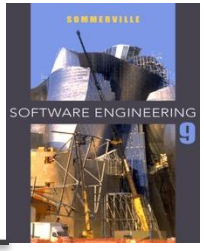
Simple? Object Access Protocol

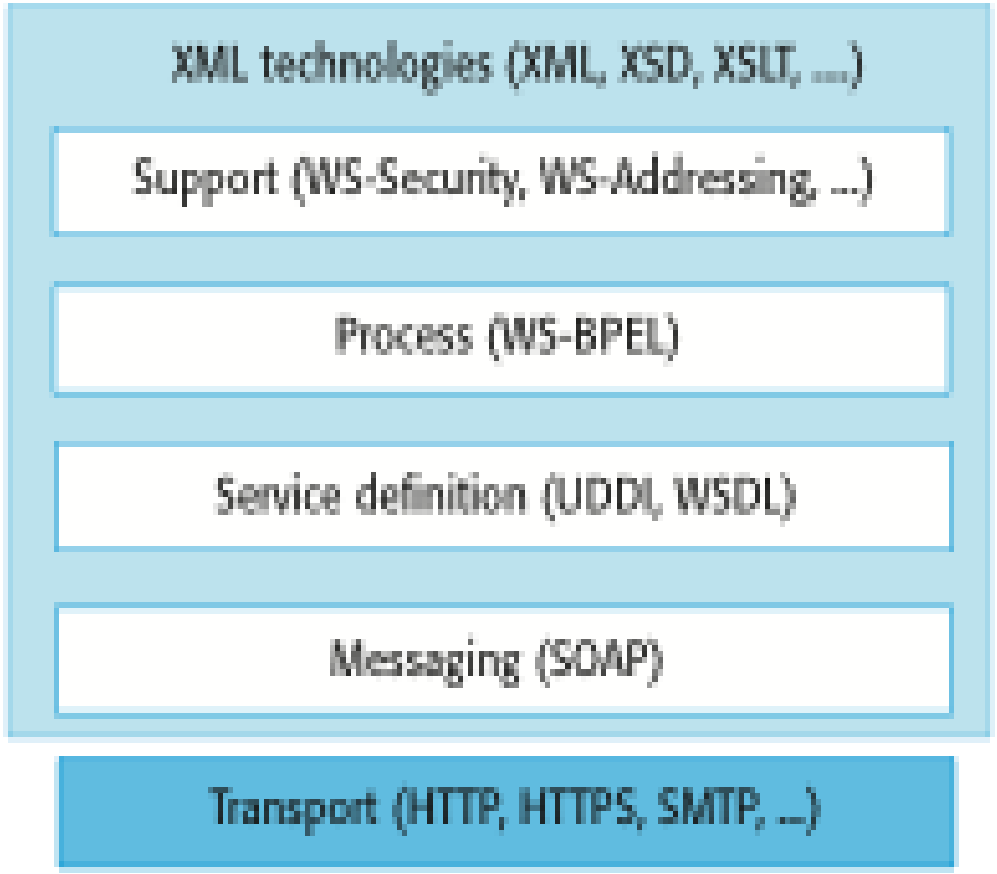


- ✧ Rather complex
- ✧ Verbose protocol
- ✧ XML even more verbose
- ✧ No standard interaction

- ✧ Fallen out of favour, too hard to implement, too slow

Web service standards

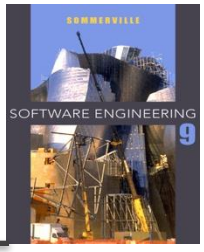




Service components in a SOA

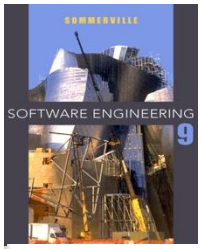
- Services communicate by **exchanging** XML **messages**
  - **distributed using standard Internet protocols (HTTP and TCP/IP**)
- A service defines what it needs from another by setting out its requirements in a message and sending it to that service.
- The receiving service **parses** the message and carries out the computation.
- On completion it sends a reply message to the requestor.
- This service parses the reply to extract the required information
- Unlike software components, services **do not use remote procedure or method calls** to access functionality associated with other services

Web service description language



- ✧ interface in WSDL (Web Service Description Language).
  - what
  - how (format of messages sent and received)
  - where (URI : Universal Resource Identifier)

Part of a WSDL description for a web service

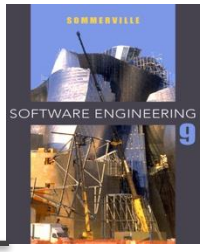


Define some of the types used. Assume that the namespace prefixes 'xs' refers to the namespace URI for XML schemas and the namespace prefix associated with this definition is weathns.

```
<types>
  <xs: schema targetNameSpace = "http://.../weathns"
    xmlns: weathns = "http://…/weathns" >
    <xs:element name = "PlaceAndDate" type = "pdrec" />
    <xs:element name = "MaxMinTemp" type = "mmtrec" />
    <xs: element name = "InDataFault" type = "errmess" />

    <xs: complexType name = "pdrec"
    <xs: sequence>
    <xs:element name = "town" type = "xs:string"/>
    <xs:element name = "country" type = "xs:string"/>
    <xs:element name = "day" type = "xs:date" />
    </xs:complexType>
    Definitions of MaxMinType and InDataFault here
  </schema>
</types>
```

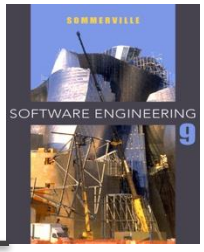
Part of a WSDL description for a web service



Now define the interface and its operations. In this case, there is only a single operation to return maximum and minimum temperatures.

```
<interface name = “weatherInfo” >
  <operation name = “getMaxMinTemps” pattern = “wsdlns: in-out”>
    <input messageLabel = “In” element = “weathns: PlaceAndDate” />
    <output messageLabel = “Out” element = “weathns:MaxMinTemp”
    />
    <outfault messageLabel = “Out” element = “weathns:InDataFault”
    />
  </operation>
</interface>
```

RESTful web services



- REST (REpresentational State Transfer) is an architectural style based on transferring representations of resources from a server to a client.
- underlies the web
- much simpler than SOAP/WSDL
- lower overhead

In RESTful services everything is a resource

POST is used to **create** a resource. It has associated data that defines the resource.

GET is used to **read** the value of a resource and return that to the requestor in the specified *representation*, such as JSON

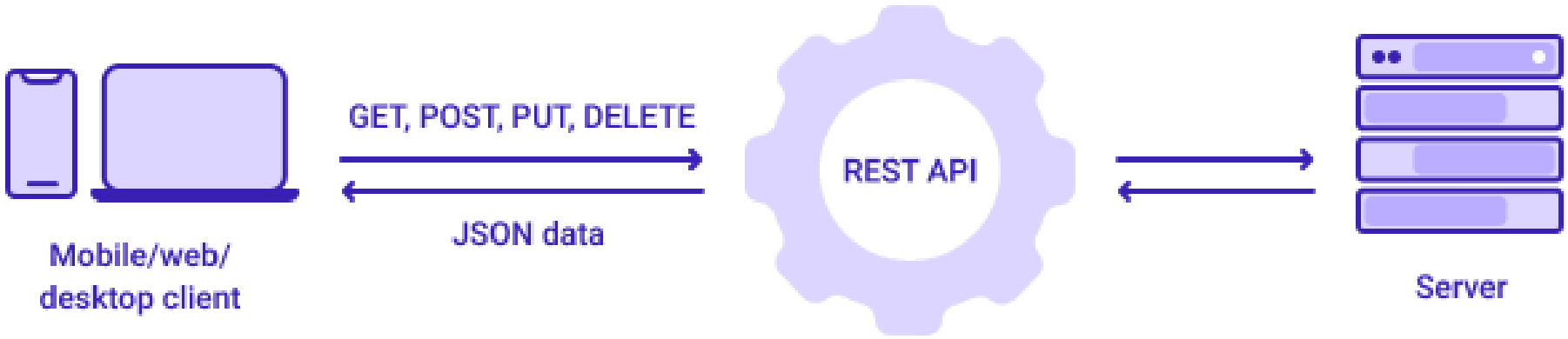
PUT is used to **update** the value of a resource.

DELETE is used to **delete** the resource (make it inaccessible).

JSON (JavaScript Object Notation) as well as XML

Stateless

REST API Model



Pros and Cons of RESTful approach

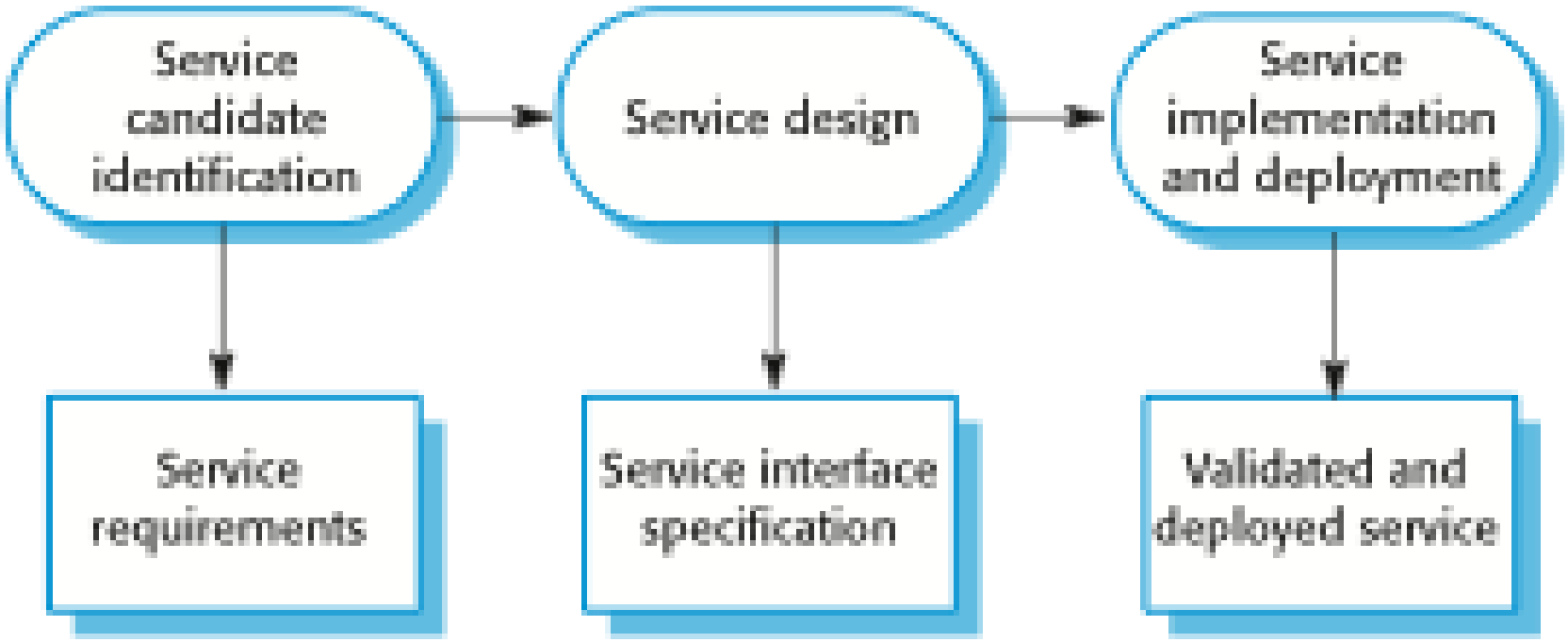
Advantages:

- Suitable for mobile
- Easy to use

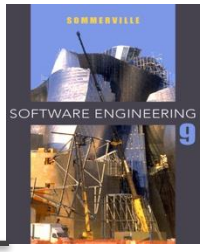
Disadvantages:

- Due to simplicity: does not scale that well
- Low level operations may complicate interface
- QoS? Reliable?

The service engineering process

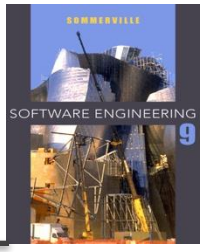


Service candidate identification



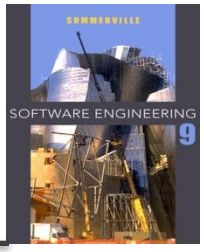
- understanding business processes to decide which reusable services could support these
- Three types
  - utility
  - associated with a specific business function
  - coordination
- Task-oriented or Entity-oriented

Service classification



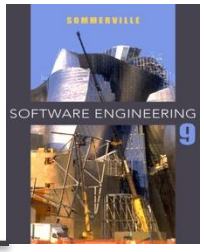
	Utility	Business	Coordination
Task	Currency converter Employee locator	Validate claim form Check credit rating	Process expense claim Pay external supplier
Entity	Document style checker Web form to XML converter	Expenses form Student application form	

Service identification



- a single logical entity used in different business processes?
- different people in the organisation?
- independent?
- must it maintain state?
- clients outside the organisation?
- different non-functional requirements?

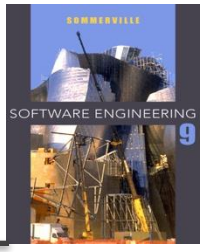
web-based procurement system



- Service requirements
  - Specific version of catalogue for each client
  - Catalogue downloadable
  - Specs and prices of up to 6 items may be compared
  - Searching facilities
  - Delivery date predicted
  - Virtual orders reserve the goods for 48 hours

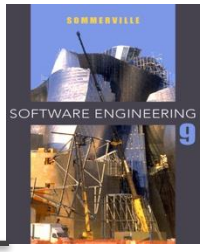
- Can that be all?

Non-functional requirements



- ✧ Access restricted to employees of accredited companies
- ✧ Prices and configurations offered to each is confidential
- ✧ Catalogue available from 0700 to 1100
- ✧ Process up to 10 requests per second

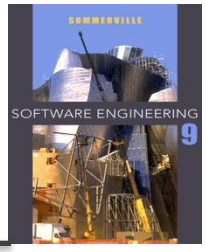
Functional descriptions of catalog service operations



Operation	Description
MakeCatalog	Creates a version of the catalog tailored for a specific customer. Includes an optional parameter to create a downloadable PDF version of the catalog.
Compare	Provides a comparison of up to six characteristics (e.g., price, dimensions, processor speed, etc.) of up to four catalog items.
Lookup	Displays all of the data associated with a specified catalog item.

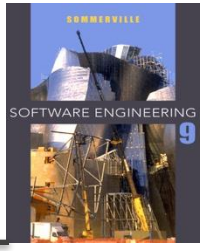
Operation	Description
Search	This operation takes a logical expression and searches the catalog according to that expression. It displays a list of all items that match the search expression.
CheckDelivery	Returns the predicted delivery date for an item if ordered that day.
MakeVirtualOrder	Reserves the number of items to be ordered by a customer and provides item information for the customer's own procurement system.

Catalog interface design



Operation	Inputs	Outputs	Exceptions
MakeCatalog	*mcIn* Company id PDF-flag	*mcOut* URL of the catalog for that company	*mcFault* Invalid company id
Compare	*compIn* Company id Entry attribute (up to 6) Catalog number (up to 4)	*compOut* URL of page showing comparison table	*compFault* Invalid company id Invalid catalog number Unknown attribute
Lookup	*lookIn* Company id Catalog number	*lookOut* URL of page with the item information	*lookFault* Invalid company id Invalid catalog number

Catalog interface design

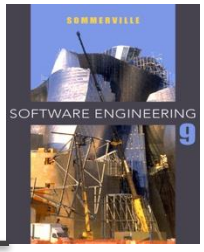


Operation	Inputs	Outputs	Exceptions
Search	*searchIn* Company id Search string	*searchOut* URL of web page with search results	*searchFault* Invalid company id Badly formed search string
CheckDelivery	*gdIn* Company id Catalog number Number of items required	*gdOut* Catalog number Expected delivery date	*gdFault* Invalid company id Invalid catalog number No availability Zero items requested
PlaceOrder	*poIn* Company id Number of items required Catalog number	*poOut* Catalog number Number of items required Predicted delivery date Unit price estimate Total price estimate	*poFault* Invalid company id Invalid catalog number Zero items requested

Documentation: service descriptions

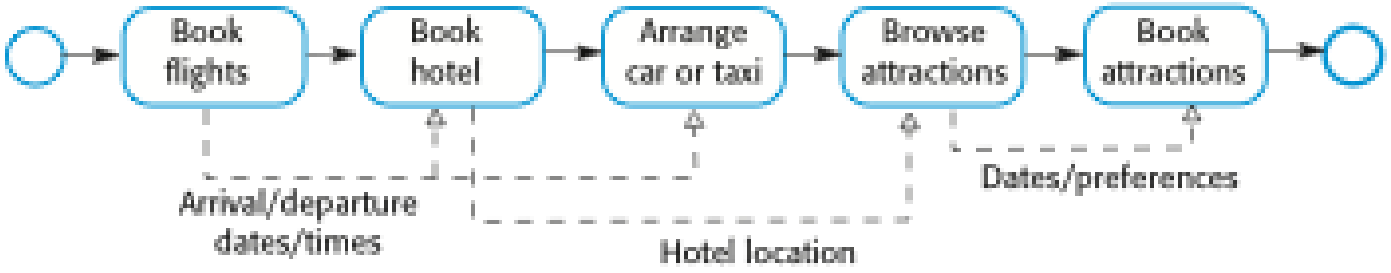
1. Information about your business, contact details, etc.
2. Informal description of the functionality provided.
3. Subscription information.
4. SOAP / REST
5. Detailed description of the interface

Software development with services

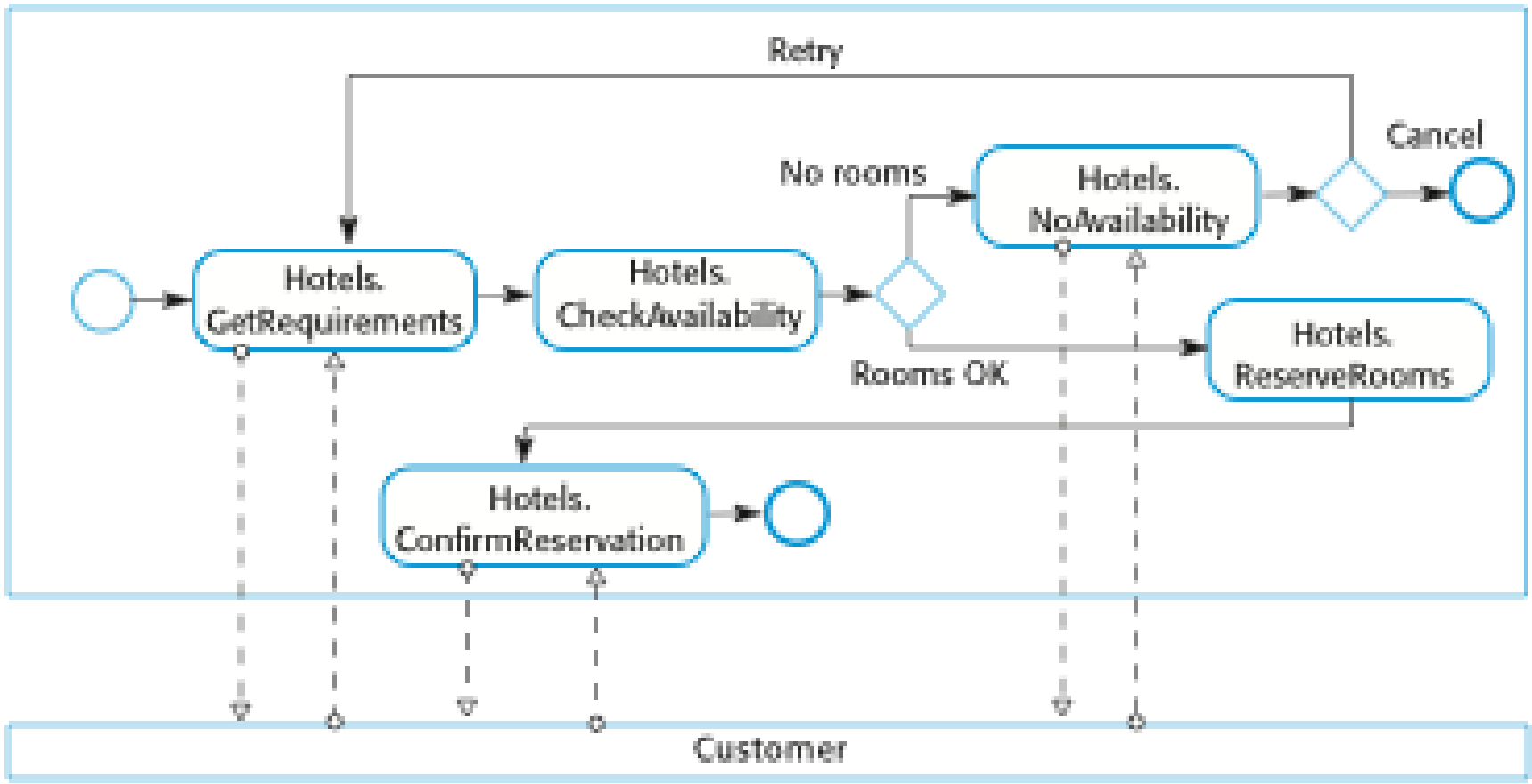


- ✧ Existing services composed
- ✧ Basis is often a workflow
  - logical sequences of activities that, together, model a coherent business process

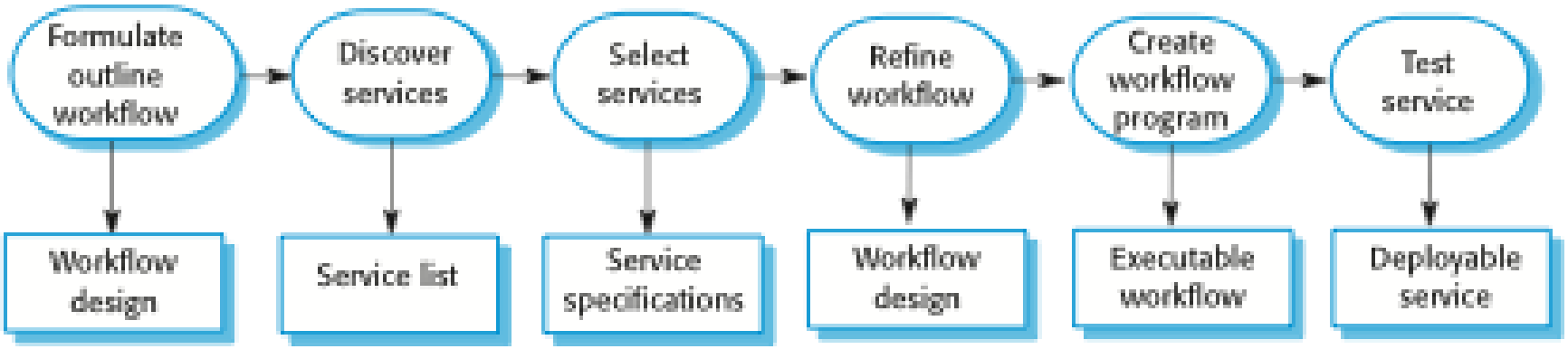
Service composition example



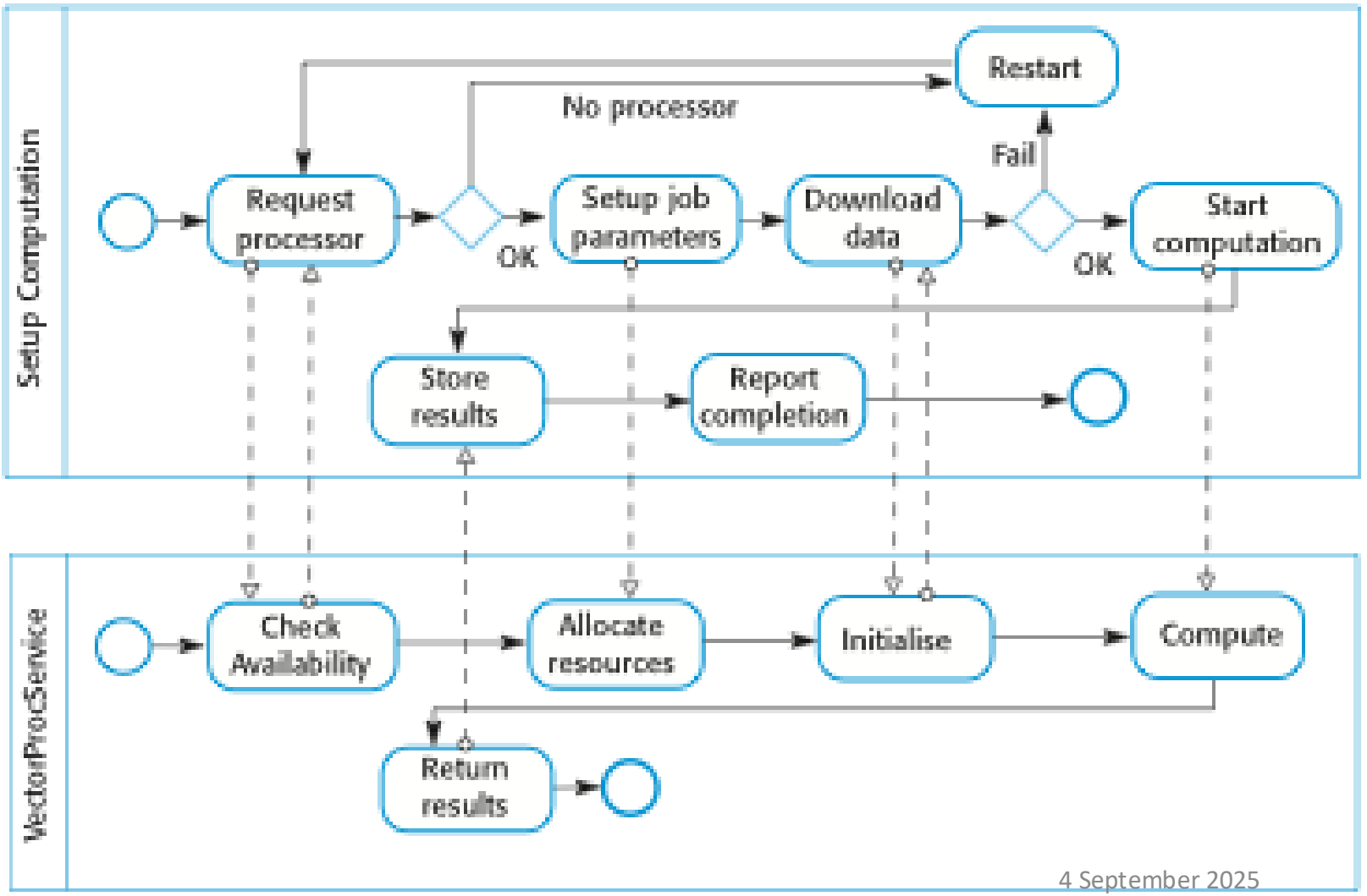
A fragment of a hotel booking workflow



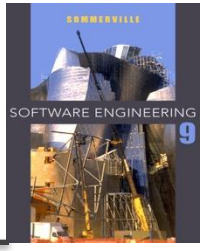
Service construction by composition



Interacting workflows



Service testing is difficult



- ‘black-boxes’
- modified by the service provider ?
- dynamic binding ?
- differing loads ?
- expensive ?
- compensating actions ?

Microservices *versus* Service-Oriented Architecture

- service based architectures
  - SOA: *share-as-much-as-possible*
  - Microservices: *share-as-little-as-possible*

- access remote services
  - Microservices: REST
  - SOA: many different protocols

Microservices Architecture Resources

- Free ebook from O'Reilly: Mark Richards, Microservices vs. ServiceOriented Architecture, 2016
  www.oreilly.com/programming/free/microservices-vs-service-oriented-architecture.csp
- Microservices: a definition of this new architectural term, James Lewis & Martin Fowler, 25 March 2014
  martinfowler.com/articles/microservices.html
- Agile coding in enterprise IT: Code small and local, Alan Morrison, 24 August 2015
  usblogs.pwc.com/emerging-technology/agile-coding-in-enterprise-it-code-small-and-local/
- Pattern: Microservice Architecture, Chris Richardson.
  microservices.io/patterns/microservices.html

SOA (Service-Oriented Architecture)	Microservices
SOA focuses on component reuse and thus less independence, tighter coupling	Microservices are stand-alone services, so low coupling
A system change requires modifying the monolith.	A system change leads you to create a new service.
Supports multiple message protocols.	Uses lightweight protocols like HTTP, REST
Better for large scale integrations and more complex applications	Better for small or web-based applications.
Less flexibility in deployment	Quick and easy deployment.
An SOA app comprised of two or three services.	A Microservices app could have dozens of services.
Uses enterprise service bus (ESB) for communication	It uses the less elaborate and straightforward messaging system
Software size is larger	The size of the Software is small